Taller: Desarrollo de APIs REST en Spring Boot con Datos en Memoria

Objetivo:

Desarrollar una API REST en Java con Spring Boot que maneje datos en memoria utilizando listas y arreglos de objetos, sin necesidad de bases de datos.

Duración: 3 horas

Requisitos Previos:

JDK 17 o superior instalado
IDE como IntelliJ IDEA, Eclipse o VS Code con Extensión de Java
Postman o cURL para probar la API
Conocimientos básicos de Java y programación orientada a objetos

Parte 1: Configuración del Proyecto en Spring Boot

Ejercicio 1: Creación del Proyecto Spring Boot

Objetivo: Configurar un proyecto base con Spring Boot

Paso a Paso:

1. **Ve a Spring Initializr y genera un nuevo proyecto con las siguientes opciones:**
    - **Project: Maven**
    - **Language: Java**
    - **Spring Boot Version: 3.x.x**
    - **Dependencies:**
      Spring Web
      Spring Boot DevTools
2. **Descarga y descomprime el proyecto.**
3. **Abre el proyecto en tu IDE (IntelliJ o Eclipse).**
4. **Ejecuta la aplicación con mvn spring-boot:run o en la clase MainApplication.java.**
5. **Abre el navegador y accede a http://localhost:8080 para verificar que la aplicación corre correctamente.**

Resultado esperado: Proyecto Spring Boot listo para desarrollo.

Parte 2: Creación de una API REST con Datos en Memoria

Ejercicio 2: Creación del Modelo y la Lista de Datos

Objetivo: Crear una clase Producto y una lista en memoria que simule una base de datos.

Paso a Paso:

1. **Dentro de src/main/java/com/miempresa/api, crea un paquete llamado models.**
2. **Dentro de models, crea la clase Producto.java:**

```
package com.miempresa.api.models;


public class Producto {
  private Long id;
  private String nombre;
  private double precio;

  public Producto(Long id, String nombre, double precio) {
    this.id = id;
    this.nombre = nombre;
    this.precio = precio;
  }

  public Long getId() { return id; }
  public void setId(Long id) { this.id = id; }

  public String getNombre() { return nombre; }
  public void setNombre(String nombre) { this.nombre = nombre; }

  public double getPrecio() { return precio; }
```

```
  public void setPrecio(double precio) { this.precio = precio; }
}
```

Resultado esperado: Clase Producto creada con atributos y métodos getter y setter.

Ejercicio 3: Creación del Controlador REST

Objetivo: Implementar un CRUD de productos con una lista en memoria.

Paso a Paso:

1. **Dentro de src/main/java/com/miempresa/api, crea un paquete llamado controllers.**
2. **Dentro de controllers, crea la clase ProductoController.java:**

```
package com.miempresa.api.controllers;

import com.miempresa.api.models.Producto;
import org.springframework.web.bind.annotation.*;

import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/productos")
public class ProductoController {
  private List<Producto> productos = new ArrayList<>();

  public ProductoController() {
    // Datos iniciales en memoria
    productos.add(new Producto(1L, "Laptop", 1500.99));
    productos.add(new Producto(2L, "Mouse", 25.50));
    productos.add(new Producto(3L, "Teclado", 45.75));
```

```
  }

  @GetMapping
  public List<Producto> listarProductos() {
    return productos;
  }

  @PostMapping
  public Producto crearProducto(@RequestBody Producto producto) {
    producto.setId((long) (productos.size() + 1)); // Asignar un ID único
    productos.add(producto);
    return producto;
  }

  @PutMapping("/{id}")
  public Producto actualizarProducto(@PathVariable Long id, @RequestBody
Producto producto) {
    for (Producto p : productos) {
      if (p.getId().equals(id)) {
        p.setNombre(producto.getNombre());
        p.setPrecio(producto.getPrecio());
        return p;
      }
    }
    return null;
  }

  @DeleteMapping("/{id}")
```

```
  public String eliminarProducto(@PathVariable Long id) {
    productos.removeIf(p -> p.getId().equals(id));
    return "Producto eliminado";
  }
}
```

Resultado esperado: API funcional con una lista en memoria en lugar de base de datos.

Parte 3: Pruebas con Postman

Prueba los siguientes endpoints en Postman:

Método	URL	Descripción
GET	/productos	Obtener lista de productos.
POST	/productos	Crear un nuevo producto.
PUT	/productos/{id}	Actualizar un producto existente.
DELETE	/productos/{id}	Eliminar un producto.

Ejemplo de cuerpo JSON para POST /productos:

```
{
  "nombre": "Monitor",
  "precio": 299.99
}
```

Resultado esperado: La API maneja datos sin necesidad de bases de datos.

Parte 4: Reto Final

Reto:

1. Añadir validaciones con @Valid y @NotNull.
2. Manejo de excepciones con @ExceptionHandler.
3. Agregar una búsqueda por nombre (GET /productos?nombre=Laptop).
4. Persistencia temporal (guardar datos en un archivo JSON o en una variable static).

Conclusión

Construimos una API REST en Spring Boot sin base de datos.
Implementamos un CRUD usando listas en memoria.
Probamos los endpoints con Postman.
Aplicamos conceptos clave de arquitectura REST.

Siguientes pasos: Integrar esta API con un frontend en Angular o React.